PRACTICAL MILESTONE

Lab 3 of 8

Search Engineering: Package, Tests, CLI, and Benchmark

Turn Labs 1–2 into a reproducible Python toolkit

Coverage. Chapter 4.
Goal. Standardise five algorithms, verify them with dataset-free tests, expose a CLI, and benchmark ten route queries.
Scenario. Convert notebook logic into reusable, testable software.
Data. The OpenFlights files and `all_experiments/utils/search_queries.csv`; unit tests use hand-made graphs.
Notebook. `all_experiments/lab03_search_engineering_worked.ipynb`; implementation and tests are in `all_experiments/utils/`.

Lab setup

Before opening the notebook, check the package from a terminal. The unit tests need no dataset and finish in under a second; the command line reproduces the Lab 2 answer:

```
cd all_experiments
python -m pytest utils -q
python -m utils --algorithm astar --start KUL --goal KEF
```

Setup cell.

```
import subprocess
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

import utils
from utils import SearchResult, bfs, dfs, ucs, gbfs, astar, haversine_km, load_openflights

ROOT = Path.cwd().parent
DATA = ROOT / "all_datasets"
RESULTS = ROOT / "results" / "lab03"
RESULTS.mkdir(parents=True, exist_ok=True)
```

Question 1 — Standardise the interface

Every algorithm must return the same fields: found flag, path, hops, path cost, expanded nodes, maximum frontier and runtime. Show the fields of `SearchResult`, then run all five algorithms on a small hand-made graph and display the results as one table.

Worked answer

```
print("SearchResult fields:", list(SearchResult.__dataclass_fields__))

toy = {"S": [("A", 2.0), ("B", 5.0)], "A": [("G", 5.0)], "B": [("G", 1.0)], "G": []}
h = {"S": 5.0, "A": 4.0, "B": 1.0, "G": 0.0}.get      # a hand-made consistent heuristic
```

```
results = {
    "BFS": bfs(toy, "S", "G"),
    "DFS": dfs(toy, "S", "G"),
    "UCS": ucs(toy, "S", "G"),
    "GBFS": gbfs(toy, "S", "G", h),
    "A*": astar(toy, "S", "G", h),
}
table = pd.DataFrame({name: r.as_dict() for name, r in results.items()}).T.drop(columns="runtime_ms")
table["path"] = table.path.map(" -> ".join)
display(table)
```

Reference output.

```
SearchResult fields: ['found', 'path', 'hops', 'path_cost_km', 'expanded', 'max_frontier', 'runtime_ms']
     found         path hops path_cost_km expanded max_frontier
BFS   True  S -> A -> G    2          7.0        3            2
DFS   True  S -> A -> G    2          7.0        2            2
UCS   True  S -> B -> G    2          6.0        3            2
GBFS  True  S -> B -> G    2          6.0        2            2
A*    True  S -> B -> G    2          6.0        3            2
```

All five functions return a `SearchResult` with identical fields, so one loop can benchmark them. On the toy graph BFS and DFS return the two-edge route S→A→G (cost 7) while UCS and A* return the cheaper S→B→G (cost 6); GBFS also picks S→B→G because $h(B) < h(A)$.

Question 2 — Separate responsibilities

Search logic, data loading, plotting and the command line must live in separate modules. List the package's modules with their one-line purpose, and the names it exports.

Worked answer

```
package = Path(utils.__file__).parent
for module in sorted(package.glob("*.py")):
    purpose = module.read_text(encoding="utf-8").splitlines()[0].strip('"')
    print(f"{module.name:<16} {purpose}")
print("\nExported names:", ", ".join(utils.__all__))
```

Reference output.

```
__init__.py      Helper code for the AI course notebooks.
__main__.py      Command-line route search (Lab 3).
core.py          Graph-search algorithms shared by Labs 1-3: BFS, DFS, UCS, GBFS and A*.
openflights.py   Load the OpenFlights airport and route files into a weighted graph.
plotting.py      Plot airport routes on longitude/latitude axes (Labs 1-3).
test_search.py   Unit tests for utils.core on tiny hand-made graphs (no dataset needed).

Exported names: SearchResult, astar, best_first, bfs, dfs, gbfs, ucs, haversine_km, load_openflights,
     route_is_valid, plot_route_paths
```

`core.py` holds the five algorithms and knows nothing about airports; `openflights.py` cleans the CSV files and computes distances; `plotting.py` draws routes; `__main__.py` is the command line; `test_search.py` holds the tests. Each module can be changed and tested on its own.

Question 3 — Run the automated tests

The tests use tiny hand-made graphs and cover start = goal, unreachable goal, cycles, different objectives, frontier updates, consistent A*, non-optimal GBFS and negative costs. Run them with `pytest` from inside the notebook.

Worked answer

```
tests = subprocess.run([sys.executable, "-m", "pytest", "utils", "-q", "--color=no"], capture_output=True,
    text=True)
print(tests.stdout)
assert tests.returncode == 0, tests.stderr
```

Reference output.

```
........                                                                 [100%]
8 passed in 0.59s
```

All eight tests pass in under a second. They need no dataset, so they run anywhere — including a continuous-integration server. The `assert` turns a failing test run into a failing notebook, so a broken package cannot produce a benchmark.

Question 4 — Exercise the command-line interface

Run `python -m utils -algorithm astar -start KUL -goal KEF` as a subprocess and show its output.

Worked answer

```
cli = subprocess.run([sys.executable, "-m", "utils", "--algorithm", "astar", "--start", "KUL", "--goal", "
    KEF"],
                     capture_output=True, text=True)
print(cli.stdout)
assert cli.returncode == 0, cli.stderr
```

Reference output.

```
ASTAR: found=True, hops=4, cost_km=11461.3, expanded=69, runtime_ms=4.20
KUL -> HKT -> LED -> HEL -> KEF
```

The command line prints the same KUL→HKT→LED→HEL→KEF route and 11,461 km that Lab 2 found: the notebook and the terminal share one implementation.

Question 5 — Benchmark ten route pairs

Run BFS, UCS, GBFS and A* on the ten queries in `utils/search_queries.csv` and summarise success rate, mean cost, mean hops, expanded nodes and runtime per algorithm. Save `benchmark.csv` and `benchmark_summary.csv`.

Worked answer

```
_, _, coords, graph, _ = load_openflights(DATA / "lab01-03_airports.csv", DATA / "lab01-03_routes.csv")
queries = pd.read_csv("utils/search_queries.csv")


def heuristic_to(goal):
    return lambda a: haversine_km(coords[a]["latitude"], coords[a]["longitude"], coords[goal]["latitude"],
     coords[goal]["longitude"])


rows = []
for query in queries.itertuples():
    h = heuristic_to(query.goal)
    results = {"BFS": bfs(graph, query.start, query.goal), "UCS": ucs(graph, query.start, query.goal),
               "GBFS": gbfs(graph, query.start, query.goal, h), "A*": astar(graph, query.start, query.goal,
      h)}
    for name, result in results.items():
```

```
        rows.append({"query_id": query.query_id, "start": query.start, "goal": query.goal, "algorithm":
    name,
                     **result.as_dict(), "path": " -> ".join(result.path)})

benchmark = pd.DataFrame(rows).rename(columns={"path_cost_km": "cost_km"})
benchmark.to_csv(RESULTS / "benchmark.csv", index=False)
summary = benchmark.groupby("algorithm").agg(success_rate=("found", "mean"), mean_cost_km=("cost_km", "mean
    "),
                                             mean_hops=("hops", "mean"), mean_expanded=("expanded", "mean"),

                                             median_runtime_ms=("runtime_ms", "median")).round(2)
summary.to_csv(RESULTS / "benchmark_summary.csv")
display(summary)

costs = benchmark.pivot(index="query_id", columns="algorithm", values="cost_km")
print("A* cost equals UCS on all 10 queries:", np.allclose(costs["A*"], costs["UCS"]))
print("GBFS is costlier than UCS on", int((costs["GBFS"] > costs["UCS"] + 1e-6).sum()), "of 10 queries")
```

Reference output.

```
           success_rate  mean_cost_km  mean_hops  mean_expanded  median_runtime_ms
algorithm
A*                  1.0      14321.11        2.2           62.7               3.39
BFS                 1.0      16469.27        1.8          323.9               3.03
GBFS                1.0      15087.33        2.1            2.1               2.90
UCS                 1.0      14321.11        2.2         2700.4               7.58
A* cost equals UCS on all 10 queries: True
GBFS is costlier than UCS on 7 of 10 queries
```

Every algorithm succeeds on all ten queries. A* matches the UCS cost on every query with a fraction of the expansions; BFS has the highest mean cost because it optimises hops; GBFS expands the fewest states but returns a costlier route on seven queries. For distance-based routing A* is the best default among the exact methods.

Question 6 — Save the evidence

Save a bar chart of mean expansions per algorithm, and note the one-line terminal command that runs the same tests outside the notebook.

Worked answer

```
ax = summary.mean_expanded.sort_values().plot.bar(figsize=(7, 4), logy=True)
ax.set(ylabel="Mean expanded states (log scale)", title="Ten-query search-effort benchmark")
ax.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.savefig(RESULTS / "benchmark_expansions.png", dpi=160)
print("Saved:", sorted(p.name for p in RESULTS.iterdir()))
print("Terminal command (from all_experiments):  python -m pytest utils -q")
```

Reference output.

```
Saved: ['benchmark.csv', 'benchmark_expansions.png', 'benchmark_summary.csv']
Terminal command (from all_experiments):  python -m pytest utils -q
```

`python -m pytest utils -q` runs the same tests from a clean checkout, which is exactly what a continuous-integration job would do. Setting up such a job is an optional extension; the required evidence is the passing local run.

Verification checklist

- All five algorithms return a `SearchResult` with identical fields.
- All eight tests pass with `python -m pytest utils -q`, without any dataset.
- The command line prints the KUL→HKT→LED→HEL→KEF route and 11,461 km, as Lab 2 found.

- Every algorithm succeeds on all ten benchmark queries, and A* matches the UCS cost on each.
- `results/lab03/` holds `benchmark.csv`, `benchmark_summary.csv` and `benchmark_expansions.png`.

Check your understanding

1. Which *one* line differs between UCS and A* in `best_first`, and which module would you edit to add a fourth mode?
2. Why is `path_cost_km` recomputed from the returned path instead of taken from the search's g value?
3. What is the `counter` in the heap tuple for, and what goes wrong without it when states are tuples?
4. Why do the tests use a five-state graph rather than the airline network?
5. The benchmark reports GBFS as costlier than UCS on seven of ten queries. Why is "seven of ten" more informative than the mean cost gap alone?

Quick-check answers

1. The `priority` function: it returns `g` for UCS and `g + h(state)` for A*. A fourth mode would be added in `core.py` only.
2. So that the reported cost and the reported path can never disagree: the cost is derived from the path that is actually returned.
3. It breaks priority ties. Without it, `heapq` compares the states themselves, which raises `TypeError` for tuples of mixed types and, even when it works, orders equal-priority entries by name rather than by the search.
4. The correct answer can be worked out by hand, a failure is easy to diagnose, and the test needs no data and runs in milliseconds — so it will actually be run.
5. A mean gap can be dominated by one bad query. "Seven of ten" says how *often* GBFS is worse, which is what someone choosing an algorithm needs to know.